

Abstract

Many organizations have adopted the Scrum framework for agile software development, but often the Scrum framework has significant administrative overhead that obstructs productivity. Many speak of the role that Artificial Intelligence (AI) could play in relieving some of that overhead, but there remains a large disparity between theoretical ideas and proven, practical applications. In this thesis, I address that theory-practice gap by investigating the technical feasibility of employing Large Language Models (LLMs) to automate core Scrum ceremonies.

In particular, this research designs and implements a proof-of-concept system for sprint planning automation, but also finding hidden dependencies, and summarizing sprint retrospectives. All this work is based upon using a Llama-3.1-8B-Instruct model, and a set of real-world project management artifacts. I designed and tested a series of prompt engineering techniques around the tasks at hand. The performance of the LLM was quantitatively performed relative to a ground truth based on past project experience.

In general, I show that the LLM has very high performance for proactive dependency awareness (F1-score of 0.89), ability to produce plausible sprint plans (favorability of 0.85), and demonstrated high performance for extracting key metrics to summarize sprint outcomes (F1-score 0.71). This thesis makes a contribution to the field by illustrating with empirical evidence, how modern LLMs can be used as efficacious social technologies to automate routine scrum tasks. It offers a practical contribution in the form of both a working proof-of-concept and a methodological template for research in future AI-augmented agile project management.

Chapter 1: An Introduction to Automating the Scrum Framework with Large Language Models

1.1 The Genesis of an Idea: A Collision of Experience, Conversation, and Curiosity

This project didn't originate in the confines of an office, sterile lab space or library, but rather an incidental telephone conversation with a friend on break in India. What started out, like many conversations, as a light-hearted catching up and reminiscing about times spent laughing together in Engineering school, along with griping about the job market trends, quickly turned into a discussion about a famous Internet meme. The meme was humorous to many due to its absurdity, because it repurposed an application of Artificial Intelligence, or AI, that would relieve some of the mounting anxiety employees were feeling by taking over its repetitive, daily work. To many, the post may have been a laughable quip, but to me it was an ignite.

This relatively paltry social media post hit home for me. I experienced the tango of modern software engineering by being in the corporate tech space for 2 years before completing my master's studies in Germany. It was this post that naïvely yet accurately articulated an itch I had throughout my entire stint within this industry, which was a constant background noise – too much admin work. All this admin work, made sense here and there, but sometimes felt counter productive to the actual making and arranging that mattered.

Of course, my friend and I ended up talking about Scrum, the predominate project management and organizational thinking within the software industry. We jokingly speculated on how much time and energy we spent on their various rituals. This potential inquiry leads us back to the “what if,” which forms the basis of this thesis, what if the ugliest aspects of Scrum could be intelligently automated? This was the problem was. The vague idea of “AI helps you with that” clarified and focused in on research effort. I envisioned a system which helped automatically prewrite sprints, which could predict when a task was dependent on another before their dependency got in the way of making progress, and which would have been able to evaluate the history of sprints to better predict the next sprints. I could already see the potential. This would not just create a time saver, it would fundamentally change what it meant to work in agile teams. It would liberate team leads and developers from this administrative hassle and let them reinvest into what were clearly wasted mental energy into doing what mattered; solving complex problems and adding value. At the same time, I was looking for a topical, impactful, and timely master's thesis topic with personal interest and contributing real-world value, so what better way to combine my personal pain point with a technological exciting solution, and this is where I narrowed my focus. The gossip becomes the catalyst for some serious research study.

1.2 The Modern Software Dilemma: The Promise and Pitfalls of Scrum

In order to understand the problem this thesis is trying to pose a background on the current state of software engineering and the role of Scrum in this state is provided. As the 21st Century began scrum ushered in a shift from set, sequential “Waterfall” project management techniques to a collection of flexible, iterative processes called “agile” (Schwaber and Sutherland, 2011). In 2001 the Agile Manifesto established valuing the people and interactions instead of processes and tools, and embracing change instead of following a plan (Beck et al, 2001).

Of the various agile methods, the Scrum Framework remains the most widely used, being used by an estimated 87% of agile teams in 2022 (State of Agile Report, 2022). This remarkable utilization is a result of scrum having a simple but robust mechanism for organizing work into time bounded iterations called “sprints” or “mini-waterfalls”. Sprints are initiated at the beginning of with a planning meeting where work is selected from a product backlog then each day these work items are discussed. Sprints unroll toward a sprint review where completed work is presented and a sprint retrospective where the process is reviewed and potential improvements are identified.

With all of this positive documentation and uses activates of Scrum can produce significant negative externalities. The rituals that can organize Scrum can in some cases become a form of transactional overhead. For example, estimates and priorities received from sprint planning can weigh heavily on the sprints themselves. On these larger projects, dependency management can turn into a tangled web of communication that can be hard to untangle. A sprint retrospective is a key part of continuous improvement but is often contingent on imperfect human memory to recall the specifics of a multi-week sprint.

1.3 Problem Space: The Gap Between Theoretical Hope and Practical Action

Once the initial excitement of identifying an interesting research area has passed, an academic grind set in. Following the direction of my supervisor, a thorough review of the literature was done to get a baseline of the landscape of AI in the context of agile. This review – which informs Chapter 2 – revealed a resoundingly clear and consistent story: the debate on the place of AI in Scrum is largely theoretical. The literature contains mostly notions of frameworks, thought leadership, and high-level ideas of what it could do, but almost entirely lacks evidence of what it has done and validated in practice.

This realization of an exceptionally significant “theory-practice gap” is the underlying gap that this thesis addresses. While academics and professionals argue that AI has the potential to fundamentally transform the entire life-cycle from user story creation to risk prediction (Perkusich et al., 2020) but almost always are not based on real-world, tested, use cases. The literature can be viewed simplistically as two streams. The first stream includes conceptual papers that contemplate “AI Scrum Masters” or intelligent agents that could manage whole projects. While these papers are interesting for exploring possibilities and creating a vision for the future, they do not provide technical guidelines or measures of success.

The second and more technical stream explores traditional Machine Learning (ML) models for specific and narrow tasks within Scrum. For example, the researchers leveraged regression models to estimate project completion timelines or classification algorithms to prioritize bugs (Choetkiertikul et al., 2018). But while they do make a contribution, they often treat project artifacts as well-structured data points while ignoring the rich unstructured and context-full nature of project text which is the true fabric of agile projects (the user stories, comments, and dialog). They can predict that a delay may happen but not provide a rationale as to why based on the semantic knowledge captured in team communications.

Whereas hardly anybody is familiar with the quickly emerging Large Language Models (LLMs) that represents a potential breakthrough and could traverse this theory-practice gap. LLMs, with their incredible ability to comprehend, summarize and write human-like texts (Brown et al., 2020; Vaswani et al., 2017), can skillfully tackle the unstructured data in agile project management. An LLM could comprehend a user story and understand the nuance of it; read through comments and understand the back-and-forth to unravel a dependency; or create a concise and coherent retrospective from sprint outcomes. This is a qualitative shift away from the quantitative analysis known with traditional ML (beyond counting) to qualitatively semantically understanding project work, which has exclusively been done by people.

This is all very new with LLMs and the benefit of being new is no clear roadmap exists for applying them in practice. Thus, the problem this thesis set-out to study is, in fact, two-fold:

1. The Implementation Gap: We must move away from theorizing and into building an operational system that uses state-of-the-art AI, namely LLM, to automate basic Scrum ceremonies.
2. The Methodological Gap: we need to develop and clearly articulate a systematic process from data to readiness all the way to prompt crafting for how to use LLMs for scrum / agile project management.

By addressing both of these concerns, this research hopes to deliver not only a meaningful proof of concept, but also meaningful and transferrable contribution.

1.4 Research Goal and Questions: A Targeted Inquiry Into the Scrum Ceremonies Using LLM.

In order to address the implementation and methodological gaps mentioned above, this present work is a focused, integrative line of inquiry. The primary research goal is to investigate the technical feasibility of using LLMs to automate (or semi-automate) the three most challenging and administratively heavy ceremonies (meetings) in the Scrum framework. This study moves from the general “proof-of-concept” they present in the paper to investigate the “how” of the implementation. The program of research will look to design, develop and test some prompt engineering to convert the unstructured, raw text from project management systems into structured, actionable intelligence.

The primary research goal will be achieved by answering three specific, but interconnected research questions. Each research question addresses a different pain point in the agile process, and will be answered with empirical data generated through real-world dataset analysis.

Research Question 1 Sprint Planning Automation

The first line of inquiry addresses the sprint planning ceremony. This kickoff meeting before each sprint begins is critical. In order to finish sprint planning, the team needs to review the product backlog and select a set of user stories, and get a commitment to scope. Spatially, the process is typically drawn out and is heavily reliant on the Scrum Master and the team's experience and instincts. As such, the first research question is:

- **RQ1: How do you create “high-quality” prompt templates for LLMs so they can generate an organized sprint plan based (partly or fully) on user stories with story point estimates?**

This research question asks if LLMs can act as intelligent planning assistants. The key is not entirely replacing human judgement, but automating the initial lengthy process of group related stories into some logical initial proposed sprint. The research then creates prompt templates that can give the LLM key properties about the issues such as issue summaries, issue descriptions, story point estimate, and priority order (probably the highest priority issue would get highest priority). With the prompt given to the LLM, he or she can discern proposed sprint based story groupings from method of inputs which would extract all of that information. The proposed grouping would then be compared to the sprintId ground truth from historical project data to see if the LLM could derive some plausible grouping of stories that could have constituted a sprintId sprint.

RQ2: Proactive Dependency Management

The second research question is predominately situated within a question of a “problem” that permeates all of all complex projects: hidden dependencies. What may be the first part of what seems above-board principled approach, could halt work which seemed simple, and by cascading effect lead to a waterfall of delayed sprints, that disrupted the real sprint on the floor. Most instances as described uses common issue description plain speak or comments references dependencies arise from in a layman's read followers of commentary style issues; it sometimes appears in space of language, and organizaing dependencies makes it more awkward, and systematic tracking troublesome. Thus the second question is stated defined as:

- **RQ2: Can an LLM discern dependencies on issue summaries and historical comments and categorize the dependent issues into a blockers, vital or not, to reduce sprint foobaore for scrum teams and sprint clarity?**

The second research question is the context of risk identification with LLM. The usage of progress ticking-tock wise to task dependencies beyond, keys words, and find an understanding of contextual understandings for the links issues as linkages task have contextual meanings. In the proposal if a user can prioritize of existence and find blocker issues from issue text, labels the studies were based on categorical variables clarified, had blockedBy, or blocks dependency from flow chart, as well as, a dependency synonymous ultimately as a description they existed in terms of the classifier relationship for dependency topics with respect to the historical project dataset. The frames of key research investigated pursurs what the LLM would be like a trying out, by means of a proactive risk ahead management tool with simply a scrum master

RQ3: Sprint Review

Finally the third research question concerns the sprint retrospective. The ceremony for constant 'increased awareness' of the retrospective. The definite adoption and inherent value of the retrospective is totally grounded on people recollecting what the he team worked in the last of the sprint. The sprint retrospective is dramatically cumulative laborious process, until memory becomes dictation. The question is to defined in terms of complement:

- **RQ3: How well can an LLM aggregate the sprint review of a little story history awareness based, using the sparse stories, data, and outcome in projections to emanate actionable items and stressing super points of emphasis?**

Here, again follows from the research assumed ground-depth LLM as an corporate, an institutional memory with contributions to a measure of what on-going knowledge capture, form foundation of the system based on history based audit. The researcher investigates, overall, we can come upon propose by propose the LLM can describe stories, different data, between issues can form summary data, walk upon measure, that use stories initiated measure and formed from measures or issues, drift in item based sprint story with initiate components that relied on providing a summarized measure of delaying item . The data scripted issues will lead to direct results into questions to serve an reproach of the research process improve measures taken away on teams improving process.

1.5 Research Contributions: Theory, Method, and Practice

By approaching the aforementioned research questions systemically, this thesis contributes uniquely and substantively to the interface of artificial intelligence and agile project management. Contributions are summarized in three dimensions: theoretical, methodological, and practical.

Theoretical Contribution:

From a theoretical standpoint, this thesis advances the scholarly conversation surrounding AI as a technology in software engineering and project management. Existing literature has focused on using "traditional" machine learning for quantitative prediction (e.g., effort estimation); this thesis is the first to examine and demonstrate how to use the natural language understanding capabilities of Large Language Models to interact with the qualitative, unstructured data that underpins agile development. It recasts the theoretical perspective from prediction, but built on understanding and generation, in terms of how AI can participate with Scrum as opposed to merely observing. It creates a new conceptual lens through which to imagine human-AI collaboration in complicated, knowledge work scenarios.

Methodological Contribution:

The central methodological contribution of this thesis is the creation and outlining of a class of innovative, task-specific prompt engineering strategies designed for the Scrum framework. The field of applied LLMs is still in its infancy, and translating a high-level, general goals into specific and actionable prompts relates to the broader question of having the AI generate the correct prompt for the intended response. This work contributes a replicable process with an example of how to prompt for three complex and nontrivial tasks: producing sprint plans, identifying semantic dependencies and summarizing sprint retrospectives. In addition, the thesis provides a structured method for assessing the abilities of LLMs on those tasks by leveraging real project data as a ground truth, which serves as the underpinning design for future investigations to bodies of knowledge.

Practical Contribution:

And perhaps the most valuable contribution, this thesis offers a substantial practical contribution in bridging theory and practice. This study concludes in an empirically evaluated proof-of-concept of an LLM-based Scrum assistant built in a Google Colab environment; this proof-of-concept is irrefutable evidence that automating core Scrum ceremonies is a real-world possibility, not some hypothetical future concept. For Scrum Masters and agile teams, this study provides a factual case for buying into AI-based tools for decreasing administrative burden and improving decision making. For developers and toolmakers of agile technology, it offers both a motivator and an initial structure to develop robust intelligent project management software solutions in the future.

1.6 Thesis Roadmap: The research expedition

The rest of this thesis is organized into four successive chapters that build coherently on each previous chapter - a progression from foundational knowledge to conclusions. The strategic approach to organizing the thesis is meant to take the reader through the research expedition from exploring the problem domain to the concluding thoughts of findings and implications.

Chapter 2: Literature Review, explores the current and previous research conduits of academic and industry literature. It reviews prior research on the use of AI and ML in agile project management, determines the current state of the art, and situates the study in the relevant academic community, thereby establishing the research gap this thesis has investigated.

Chapter 3: Method didactic details an explicit, transparent outline of the research design and methodology. This chapter discusses the type of data utilized for the experiments, the architecture for the LLM-based system, provides a comprehensive breakdown of prompt engineering applied to each hypothesis, and identifies the criteria used to measure the responses of the model.

Chapter 4: Results and Discussion depicts the empirical results from the research methods. Reports the performance accuracy of the LLM to perform the artificial tasks of sprint planning, dependency recognition, and retrospective summarization. This chapter goes beyond just reporting data; it also includes discussion, interpreting the results concerning the hypotheses, and cross-referencing the research with previous literature.

Chapter 5: Conclusion provides closure to thesis, summarizing the body of the research project. It summarizes the primary results considers theoretical and practical implications of the research, openly discusses the limitations of the study, and proposes opportunities for future research in the emergent field of AI-based agile project management.

Chapter 2: A Detailed Examination of AI and Large Language Models in Agile Project Management

2.1 The Necessary Union: From Agile Philosophy to AI-Supported Project Management.

Agile project management, which was articulated in 2001 with the publication of the Manifesto for Agile Software Development, is one of the most transformative paradigms in software development history (Fowler & Highsmith 2001). Agile project management was created in reaction to what were seen as failures of the traditional, linear "waterfall" methods that had been used for generations. These models were slow to change and took too long to develop, which meant they couldn't keep up with today's fast-paced way of software developing. The Agile Manifesto provided an alternative set of values: individuals and interactions over processes and tools; working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan (Beck et al. 2001).

Although agile is often referred to as a methodology, Scrum is widely considered one of the frameworks that resulted from the Agile movement; recent reports indicate Scrum is used by as many as 87% of agile organizations. One of the reasons for this wide use is Scrum's deceptively simple, iterative process. Work is organized in time-boxed "sprints," typically lasting 1–4 weeks timeframes where each sprint is a self-contained project and product increment that can be delivered quickly. This iterative cycle of planning, doing and checking aims to introduce fast feedback loops, incremental improvements, and an ongoing pace of progress (Schwaber & Sutherland, 2020).

At the same time, the very success and acceptance of Scrum has revealed a large tension. As software projects have enlarged and their complexities evolved, the overhead associated with the proper implementation of Scrum has significantly intensified. The five core ceremonies of Scrum (sprint planning, backlog grooming, daily stand-ups, sprint reviews, and retrospectives) are more than necessary - they are resource intensive!!! In particular they require significant amounts of time and cognitive energy from the team. Barua et al. (2025) suggests that development teams may lose 20%-40% of productive capacity due to these overheads. Herein lies the paradox - a framework designed to create agility, can further create a drag to agility – a need for a lightening fast use of Artificial Intelligence (AI) automating outdated administration burdens.

So, the merger of AI and agile project management is not a case of simply jockeying for novel technology, but a matter of making do. The original versions of this researched used AI simply as a way to improve human capability and flow. Friends of BOSS and a BOSS research product called the Project Engine seek to use automated AI powered tools to better have team collaboration and engagement, especially for distributed or hybrid work arrangements by providing better collaborative communication with a singular source of truth (Parouha & Prasad, 2025; Uysal 2022). AI can be applied to produce better estimations and better make decisions. Use of the information created by AI is extensive and complex (Das, 2025).

With the development of DevSecOps, its not about melding models with agile, its about integrating development-securing-operating. AI is now an integral enabling for an improved secure, efficient and innovative SDLC (Pakalapati et al., 2023)! ai models to review your code for security flaws while your updating is like forecasting a crystal ball untangling integrations that could have develop, think massive testing and organizations pipeline, aligned with agile, to delivery value(s) fast and securely (Camacho, 2025).

But as you will see in the next sections, the integration of AI and agile frameworks, has further been an evolution process. The robust promises and possibilities of AI have continued to evolve over time, leading to the emergent development of a different “waves” of research and practice. This tale of literature, perhaps tells two sides of the same story, the road map from simple, data predictions to more complex, and language automation.

2.2 First-Wave Automation: Traditinal Machine Learning in Agile Project Management

The initial use of Artificial Intelligence to solve agile project management issues was initiated by traditional machine learning (ML) methods. This first wave of automation, which began in the late 2010s, aimed to utilize the predictive ability of statistical modeling on the structured data output that agile software development generated. Project management software such as Jira, Azure DevOps, or Trello became unintentional data gold mines, gathering task, bug, and user story data at a granular level. This data also included story points, issue priorities, assignee history, time-to-resolution, sprint completion rates and so forth and served as the basic input to a new generation of data-driven project management tools. Castillo et al.'s (2024) systematic literature review provides a full discussion of this era and notes that research efforts were almost exclusively focused on three types of ML approaches- regression for forecasting, classification for risk identification, and clustering for classification and discovery.

Regression for Forecasting and Estimation

Estimation and/or forecasting remain some of the most difficult aspects of agile development. Whether estimating how much work a team can do in this sprint (sprint velocity), or estimating how long this project will take is important for stakeholder management and planning. Regression analysis became the leading approach for addressing this challenge. Researchers utilized models from linear regression to more complex algorithms such as Support Vector Regression (SVR) and Gradient Boosting Machines (GBM) to estimate these important performance indicators. For example, Palii (2025) demonstrates using historical sprint data - team size, number of issues, and total initial story point estimates - to create a model that predicts the expected velocity of future sprints. This shift allowed project managers and Scrum Masters to - instead of blind, "gut-feel" planning - to have a more data-driven approach and, thus, have more realistic commitments while managing expectations for those participating on the team. The Agile Scrum Sprint Velocity Dataset, an open-source dataset project using sprint data, including from studies like Spring XD and Apache Meso, have been instrumental in benchmarking such prediction models (Randula Koralage, 2020).

Classification for Risk Management and Quality Assurance:

While panellists used regression for questions including "how much?", and "how long?" - , classification models were needed for, "what if?"). Specifically, these were for risk management and quality assurance. Much research effort has gone into defect prediction, which is an important topic in any given software engineering project. For example, if you can train a classification model on historical data from repositories like the NASA PROMISE Software Engineering Repository – which has defect logs in a wide breadth – you would be able to predict if a piece of code or some user story would likely contain a bug (PROMISE, c. 2010s). The features that were used in these models were varied, including code complexity metrics (e.g., cyclomatic complexity), how many times a file had been changed prior, and the experience level of the developer who wrote the code.

In addition to defect prediction, classification was used for automated issue triaging. With large projects, there can be an overwhelming amount of incoming bugs and feature requests. Goyal and Gupta (2025) examined classifiers to automatically assign a priority (e.g., critical, high, medium, low) to incoming bugs/issues based on features generated from their description, reported severity, and number of dependencies. Automated triaging can help the team leverage their resources and address the most important work first. Sarwar and Imtiaz (2025) and Shah and Yadav (2025) extended this research by integrating ML into the software testing as well. This integration increased the usefulness of the automated testing suite and decreased the reliance on manual QA.

Clustering for Backlog Organization and Theme Discovery:

The third and final major category of traditional ML application involved leveraging unsupervised learning, with clustering used to attempt to impose order on the chaotic product backlog. A product backlog can include hundreds, if not thousands, of user stories, such that it's difficult for a Product Owner to see the forest through the trees. Techniques such as k-means clustering and DBSCAN were used to cluster related user stories in the backlog via the summaries and descriptions' semantic similarity. These algorithms grouped in the backlog using NLP techniques, like TF-IDF (Term Frequency-Inverse Document Frequency) or word embeddings (e.g., Word2Vec) to turn the user story text into numerical vectors - allowed to identify latent themes and epics in the backlog (Guo, 2025). This grouping made it easier for Scrum Masters and Product Owners in backlog refinement sessions to prioritize related blocks of work, then focus on the structure of the product roadmap.

Limitations of the First Wave:

With these notable successes came serious drawbacks in this first wave of automation based on ML that were only unmasked over time as the field matured. The major limitation was reliance on highly structured data and engineered features. Models may have predicted that something was high-risk, but had no deep semantic intelligence to characterize why based on the nuanced description and options thread content of the issue. The rich contextual, and ambiguous, natural language posed by teams members - was largely persistently lost in translation as numerical features (Castillo et al., 2024). Adding to this disadvantage, many

models are “black-box”, and complex, difficult to interpret, which resulted in distrust to base important project decisions from a model prediction individuals cannot comprehend (Das, 2025). Lastly, these supervised models were very reliant on having large, quality labeled datasets for training which creates a barrier to entry for smaller organizations or newer projects without historic datasets. These limitations gave rise to the intellectual and practical need for a new suite of models to engage directly with the project management native language which ushered in the existence of LLMs.

2.3 The Paradigm Shift: The Rise and Technical Foundations of Large Language Models (LLMs)

Contacts' inherent constraints of previous machine learning to process complex, unstructured, and sometimes uncertain language of agile project management identified a pressing and immediate requirement for a different category of AI models. The paradigm shift into development of Natural Language Processing (NLP) through Large Language Models (LLMs) satisfied this requirement. Notably, the growth of these models, specifically within a transformer architecture, is not an incremental improvement, but a paradigm shift of how machines converse and understand human language.

When tracing the historical lineage of this paradigm shift, one can observe anterior to NLP models such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs), which were developed from sequential processing models. These architectures were groundbreaking in their day, but struggled with long-range dependencies, or the ability to relate distant words and concepts in a text. This was especially difficult due to the long document context attribution typical of project management artifacts. The watershed point was achieved from the following research article "Attention is All You Need" Vaswani et al. (2017), that introduced with the transformer. Instead of relying on recurrence, a transformer, which relied on a self-attention mechanism. This model's attention mechanism can therefore allow the model to dynamically weigh the relevance of different words in the input text when it comes to some word which effectively allows the model to build a somewhat sophisticated, context-referential representation of the entire text at the same time. This capability to capture complicated, long-range dependencies is the real secret sauce that opened the door to truly large-scale language modeling.

After this work, an entire new generation of models emerged, most famously, Google's BERT (Bidirectional Encoder Representations from Transformers) (Devlin et al., 2019), and the OpenAI GPT (Generative Pre-trained Transformer) family (Radford et al., 2018, 2019), with both a small structural change (replacing the transformer decoder with a transformer encoder) and to pre-train on much larger size datasets (Hugging Face Datasets does publicly available datasets in the range of 0.6 to about 1.7 billion examples). They are large in terms of multiple notions of “large”, they have an extremely large number of parameters (usually presented in hundreds of millions and nearing one trillion), they train on datasets that have massive, internet scale text and code corpora, and they require a high powered infrastructure to train or deploy. It is not only the sheer scale that powers their extraordinary emergent properties.

To train these models they typically go through two stages. The first stage is pre-training, which we characterize as unsupervised, and the model is trained on a large dataset, and the model's

objective is extremely simply to predict what the next word in the sequence will be, or to fill in a masked-out word. At this stage, the model is used to learn a model based on the statistical structure of the language, which in turn if properly trained can learn, grammar, syntax, facts, and even basic reasoning abilities. The second stage of the process is fine-tuning, a supervised learning stage on data from a smaller, task-specific, labeled dataset on which the model that was pretrained is further trained on. This fine-tuning adapts the general-purpose model to a specific application in nature such as, text summarization or sentiment analysis. More recently, approaches like prompt-tuning, and in-context learning have come about, which allow practitioners to simply direct the model's behavior using specific crafted instructions (prompts), and a few examples, and often negates the expensive fine-tuning process (Brown et al., 2020).

It is this unprecedented combination of scale, architecture, and training approach that makes LLMs so suitable to the challenges of agile project management. E.g., unlike traditional ML that requires structured, numerical input, LLMs can take raw, natural language text as input. In fact, in the many cases of agile management, the LLM can dig much deeper as it essentially consumes that information directly and can read and understand the content of a user story, a bug report, and comment thread, etc., in it's natural or native format (at first) preserving that semantic richness or context that is usually discarded in feature engineering. A more complete study by Kumar (2024) contains a taxonomy of LLMs, characteristics of various architectures, learning paradigms, and applications, and summarize the visible challenges of utilizing LLMs.

Thus, LLMs do not represent the data where input is from a table of numerical features, but analyzing language itself as the data source, which represent a paradigm shift in perspective allowing for a more simplistic and natural means of interacting with the AI. Similar to how a project manager can engage with a human assistant as part of a team: by asking questions, and providing instructions, direction, etc. in natural language. Investigated as a use case in studies by Shahriary et al. (2025) and Mirjalili (2024), and LLM can be prompted with the following task instruction 'summarize the key blockers that were discussed in this sprint retrospective,' or 'develop three user stories based on this high-level feature description'. Compared to the traditional MTs or MLs where direct, programmatic instruction was necessary for their input.

The generative capabilities of these systems present an almost limitless form of automation. It can analyze and classify existing information and create new content. This is a cardinal capability of drafting the first version user stories, acceptance criteria (Karpurapu et al., 2024), sharehold summary reports, etc. With the findings of the research by Fan et al. (2024) titled 'WorkflowLLM', we see these utilized trending towards depicting systems as 'agents,' where these LLMs can 'orchestrate' complex, multi-series workflow with attached autonomy lending to an indirect dismantling of the human Scrum Master or AI assistant. The processed basis of being able to comprehend language and generate language is the ultimate goal of achieving this new paradigm beyond this integrated for support of developing small-scale workflows vs (ML or traditional), more calculations or algorithms. Taking the potentials of language understanding and generation presents a shift of holism on the approach to AI augmented agile project management, vs flexibility of support and simply using mapping processes.

2.4 Main Uses of Large Language Models for Automating Scrum Ceremonies

The theorized capability of Large Language Models, as demonstrated in the previous section, has its clearest value proposition in practical applications. The agile Scrum framework, through its use of natural language artifacts and ceremonies that require heavy communication, is prime territory for LLM automation. The research literature in this domain has recently shifted from speculative considerations to preliminary implementations. Current research is converging on the automation of the most cognitively intensive and administratively onerous portions of the Scrum Master's work. This section provides a review of the main uses, aligned with the three main pillars of this thesis: sprint planning, dependency and risk identification, and retrospective review.

Automating Sprint Planning and Backlog Refinement

Perhaps the most significant ceremony within the Scrum framework is the sprint planning meeting because this is the point where the team technically commits to the work and sets the overall course of the sprint. However, it is also one of the most complicated and time-consuming parts of the process. Topics for the meeting involve: digesting the product backlog, compromise on priorities, estimating effort (usually through abstract measures such as story points), and assessing team capacity. Each team member and scrum master can only process and engage with a finite amount of information at once.

The research literature shows that LLMs lend themselves exceptionally well to supporting and, at times, even automating parts of this process. A Magna Cum Laude study by Shahriary et al. (2025) explicitly studied the application of LLMs as "AI Scrum Master", from project planning. Their study indicated that by supplying an LLM with a set of user stories and their associated metadata (e.g., story points, priorities, issue types), an LLM would generate a reasonable, sensible sprint plan, clustering stories together in a fashion similar to what experienced human project managers would do. The secret was to use highly sophisticated prompt engineering - in fact, the input to the LLM was explicitly crafted to direct the model's reasoning. The results provided compelling evidence that LLMs could automate the arduous task of sorting and combining backlog items into a draft sprint plan to be adjusted by the team.

Following this study, other studies have begun to investigate the upstream task of user story creation and refinement. It is key in the path of accurate planning to start with a concise user story with rigorous acceptance criteria. However, it takes time and effort to draft good user stories. Li et al. (2024) document how LLMs can act as generative agile assistant by utilizing a high-level feature request and producing a draft set of user stories with excellent hierarchical structures and using the traditional "As a [user], I want [feature], so that [benefit]" construction.

Additionally, LLMs have demonstrated amazing success in generating acceptance criteria, which represent the "tests" in Behavior-Driven Development (BDD). BDD eliminates ambiguity by presenting requirements as a set of testable scenarios. In a detailed study conducted by Karpurapu et al., (2024), they considered the ability of several leading LLMs (including GPT-4, Llama-2, and PaLM-2) to produce these acceptance tests from user story descriptions. Their investigation identified that particularly with "few-shot" prompting (which

provides the model with a couple of examples of the desired output), that these models can generate high-quality, relevant acceptance criteria. This automation speaks to a key bottleneck in the “definition of ready,” which ensures that the user stories are sufficiently defined, and hence testable, before they are accepted into a sprint. The workflow presented by Li et al. (2024) (see Figure in Section 2.2) also explicitly denotes the opportunity for LLMs, to automate the collaborative loop of reviewing and updating acceptance criteria.

Proactive Dependency and Risk Detection

A sprint can be less about the perfect plan and more about risk management. One of the most insidious risks of any software project is the untraceable potential of task dependencies. A user story can appear tractable until it employs some condition that is dependent on the completion of another task. Not limited to within your current sprint, maybe this work is from another, or even some work from another team, now you are blocked, creating a ripple effect of delay. Project management tools allow you to analyze issues to create explicit links. However, these links are manually sourced and are often out of date.

This is where the semantic depth of LLMs give them a definitive advantage over traditional. The nuanced signals of dependency are often hidden in the natural language of issue descriptions and, more importantly, in the threads of comments where developers interact and debug. A developer might comment, “I cannot work on this until the new API from Team B is deployed,” or “This appears to be related to the defect we experienced with the authentication service last month.” A traditional ML model, not to mention a keyword-based search, probably would not comprehend the importance of such predictions.

The work of Konrad et al. (2024) on “cognitive agents” using LLMs shows how these models can be trained to ingest and work with this communication to find blockers and dependencies that have not been formally recorded. An LLM can make meaning from the semantic meaning of the text and, thus, recognize dependencies across issues, flagging them for the Scrum Master, and rank ordering them as “critical” or “non-critical.” This shifts the role of the Scrum Master from a reactive problem-solver to a proactive risk manager. The model outlined in the paper by Shahriary et al. The research exempts it claims Expo et al. (2025) builds upon, recognizing an LLM can be asked to not only plan the sprint but also to “read all comments and identify any potential risks or blockers listed”. This pre-emptive detection enables teams to mitigate blockers in advance of the sprint planning, e.g. it can rearranging the backlog or converse with another team, instead of being blocked mid-sprint.

The Intelligent Retrospective and Continuous Improvement

Sprint retrospectives are the kick parent for the agile value of continuous improvement (kaizen). It is where the team collectively reflects on the previous sprint - the highs, lows and what will improve the next time. The utility of the retrospective is hampered by the shortcomings of short-term memory and although they are valuable, they rely on subjective, anecdotal record evidence.

LLMs potentially are an absolute game changer for groups to pivot, factual, data evidence for these conversations. If an LLM summarizes the team’s complete digital record of a sprint -

sprint knee commitment, final resolved issues, and the issues story point estimates ("story point drift"), number of NEW bugs, comments, activity logs - the LLM could deliver a short, evidence-based summary of the sprint. Karpurapu et al. (2024) reference this, saying that the summarization capabilities of LLMs will essentially be a low-hanging fruit for knowledge capture.

Rather than the team starting the retrospective with respect to "what do you all remember?" an AI-generated summary stating "In this sprint, the team resolved 85% of the story point story commitment. The velocity was 10% lower than the three sprint average. There were three user stories carried over to the next sprint, and the problem had to do with two of the same outside dependency first articulated in comments of ticket XYZ on the third day of the sprint." Not just simply what the retrospective review is – it takes away from subjective memories, and allows those memories to take meaningful dialogue as evidence-based problem solving.

Research is currently furthering this. Fan et al. (2024) with "WorkflowLLM" and Mirjalili (2024) with agile optimization frameworks show promise of a more independently "agented" system. This system could not only summarize the retrospective, but also look at trends from several retrospectives to identify recurring problem impediments, and perhaps recommend exact, process improvements for the team. This would be the ultimate goal for AI-augmented agility; to engage in learning systems to help teams not just do things more efficiently but also get better over time.

2.5 The Expanding Horizon: Agile and AI beyond the Limitation of Software Development

The disruptive power of aligning agile with artificial intelligence is not contingent on the software development lifecycle. In fact, the essence of agile—iterative development, fast feedback loops, and flexibility—is inherently useful in any space with high complexity, uncertainty, and continual change. As the cognitive overhead of managing this complexity is increasingly automated via AI—especially LLMs—these enhanced agile practices are advancing to a host of engineering and industrial space. The ramifications of this movement offer greater context for the work being visited in this thesis. The outcomes being formed for Scrum automation will have significance well beyond the software sector.

Agile Hardware and Semiconductor Design

The most astonishing demonstration of this cross-domain fertilization is in electronic design automation (EDA) and semiconductor fabrication. Traditional integrated circuit (IC) design is among the most complex engineering problems tackled currently, with millions of lines of hardware description language (HDL) code and elaborate verification. Up until recently, the industry has been dominated by strict waterfall type design cycles that extend for years. The increasing complexity and the need to drive out time-to-market have presented a very compelling case for agile.

The literature points to LLMs as a likely critical driver of this shift. Firouzi et al., in 2025, and Xu et al. (2025) provides a thorough survey of the use of LLMs in EDA. For example, LLMs

may be used to create synthesizable Verilog or VHDL code from high level, natural language explanations, just as application code can be produced from user stories. This offers hardware designers a way to test various architectural options much faster. LLMs are also being used to get some aspects of the verification process automated, oftentimes the longest period of time in chip design. ChatCPU, discussed by Wang et al. (2024) are examples of LLMs that help engineers to create verification testbenches, and to debug the simulation failure including a log file analyzer, and even help with synthesis placement more efficiently bound to keys. LLM methods are reducing these tasks to shorter design iterations, which is the ultimate goal of agile design method with hardware cores, and apply the ability to “fail fast, learn fast”, agile heart in the little & compact world of hardware.

Intelligent Industrial Automation and Manufacturing:

The manufacturing industry offers another rich domain of application for AI-enabled agile principles. Manufacturing is shifting away from mass production of homogeneous parts toward multiple, flexible, custom production runs. This puts demands on the factory floor, and supply chain process to flexibly react to demands, as a central principle of agility.

This entryway is focusing LLMs to create increasingly smart and flexible industrial automation. Xia et al. (2024) explore LLMs as ways to control industrial robots/machinery via natural language commands. They significantly decrease the complexity needed to reprogram a production line as the factory floor manager could, in principle, tell the system to “reconfigure assembly line for product model B”, instead of relying on a copy of the assembly line as input or a mechanistic programmer who could not accept explicit or implied language as instructions. The ability to quickly reconfigure and adapt is a key feature of agile manufacturing.

Outside the factory floor, LLMs are enhancing efficiencies of adjacent manufacturing services. Wulf and Meierhofer (2025) describe application areas such as technical documentation management where LLMs summarize lengthy maintenance manuals or respond to FAQs entered by a field service technician, again through language. They are utilized to analyze collection reports from customers to identify common problems, and quality control monitor reports to find/fix repeated issues, and provide updates to the manufacturing process creating a feedback loop to sustain continuous improvement.

Autonomous network and infrastructure management:

The management of large scale, critical infrastructure such as telecom and optical networks face similar total complexity challenges. These networks produce huge flows of real-time performance and telemetry data from heterogeneous devices. Monitoring, and reacting, to this data is practically infeasible.

Cruzes (2025) offer a compelling case explaining how the transformer architecture used in LLMs are ideally suited for complex, sequential data streams. They describe how LLMs could use the flow of network data as “language” and then train these models to be able to predict the patterns leading to failure. In this way one would move from reactive to predictive maintenance mode quickly, which is a desirable agile practice. An example LLM could monitor a deteriorating optical signal on a trans-Atlantic cable, map it to a recent code release on an

associated piece of hardware, and automate the sending of a message back to a network operations center with a diagnosis of the problem and a suggested plan. An example of operational agility, enabling providers to sustain higher reliability and performance by acting to issues, not from failures as closely related to consumers/patients.

In summary, the potential use of AI-augmented agile practices has been extrapolated into other domains suggests a more widely applicable reality – the fundamental challenges of complexity management, optimizing innovation, and managing changes are not limited to software. The methods and technologies that have become emphasized in agile software practices, primarily the utilization of LLMs in order to automate cognitive and managerial tasks, have proved to be effective across a broader spectrum than anticipated. Hence, the key studies in this thesis about the case of Scrum is contributing to a broader framework of comprehending the interplay of humans and AI working together towards solving problems of complexity in the 21st century.

2.6 Tackling the Limits and Ethical Issues of Large Language Models,

Integrating Large Language Models into agile project management is an exciting prospect, but it presents significant challenges that require thoughtful and critical consideration. An exhaustive literature review should not conclude without a measured review of the technical limitations, ethical challenges, and pragmatic concerns that come with Large Language Models. To move ahead without acknowledging these challenges would be to exchange technological ability for precise judgment. The research highlights multiple areas of concern that must be minimized to ensure Large Language Models can be rightly adopted and utilized.

Technical Limitations and Reliability Issues,

The most well-known technical issue with Large Language Models is the phenomenon of “hallucination”—well-structured, grammatically sound, and contextually baseline-appropriate text generated, but factually wrong, absolute gibberish, or disconnected from the input source (Kumar, 2024). The Large Language Model is not some kind of database of facts; rather, it learns statistical correlations about language. Therefore, these types of models will (always but usually) create information with high availability and unfortunately, too often they create information with high availability. The risk of hallucinatory behavior in project management is vital. If a Large Language Model hallucinates a task’s dependency, misinterpret a stakeholder’s need, or instances construct added value in sprint retrospective, these can cause deadlines, resource displacement, and ultimately, a loss of trust in the workflow. The inherent unreliability necessitates that LLMs in this capacity - for the foreseeable future - be able to operate in a “human-in-the-loop” way, and the outputs would be treated as drafts or suggestions that a knowledgeable human would verify, rather than as definitive instructions.

A second associated technical difficulty is the question of domain specificity. As pointed out by Tizaoui and Tan (2024), almost all large LLMs are pre-training on large general-purpose corpora of public internet text. This generates a vast range of knowledge, but does not develop domain-specific, deep, technical knowledge required in many such professional domains. An LLM might understand the meaning of “user story” in general terms, but is unlikely to know the technical architecture of the project it is responding to, the communication style of the team,

or the organization's unstated priorities. This domain-specific context is what makes an LLM output triable, shallow or just plain useless. The remedy here is sophisticated strategy by ways like fine-tuning the model on a private domain-specific dataset (which is expensive and intensive with data) or employing very technical, context-sensitive prompt engineering strategies which is one of the research engaged with in the thesis.

Finally, the computational cost of the models represents a challenging practical hurdle. Training a state-of-the-art LLM is a colossal computational, energy, and monetary cost after being initiated, putting it entirely out of reach except for largest technology companies (Kumar, 2024). The resources spent running inference (i.e., using a pre-trained model) raises questions about the cost effectiveness and environmental impact of using these models for common, everyday tasks, particularly by smaller organizations.

Ethical, Privacy, and Security Issues:

In addition to the technical limitations, there are a variety of ethical, privacy, and security issues by deploying LLMs in an organizational context. An AI Scrum Master to be of value would need access to a firehose of project information and data, including source code, strategy documents, org-relations, business plan, even the nonprofessional and candid communications between team members. This information is often highly sensitive and proprietary. If any of this is sent out to a third-party API VM to process, this management could lead to a data breach, including IP theft, and be noncompliant with GDPR, HIPPA, or potentially CCPA (Kumar, 2024).

This risk is a significant barrier to potential uptake of cloud-based LLM solution, and is generating more interest in smaller, local, open-source LLM that can run on premises, often requiring a performance tradeoff.

Another significant ethical issue with LLM tech is the potential for broader societal biases to come forward with these models. For example, if an LLM is trained on biased texts from the internet, it would till commonality of factors, and therefore potential outputs could even marginally benefit people from a specific demographic more than others in a retrospective summary. Even as simple as assigning technical types of tasks to one gender vs another. Such biases—even those that are unintended—can be very damaging to team morale, psychological safety, and the values of diversity and inclusion. Addressing this requires ongoing efforts around data selection and identification of bias and continual checking on the outputs of the model.

Societal Considerations and the Future of Work:

The emergence of the 'AI Scrum Master' raises some societal considerations related to the future of work and the potential for job losses. While there is plenty of literature with a utopian view of AI that assists but does not take over the human project manager, there are economic incentives to automate and to reduce the workforce. Thus, there is an important area of discussion around the expected long-term implications of AI towards a profession in project management. Another emerging issue is deskilling. If we position our project manager resources heavily on AI to perform planning, estimating, risk examination and other complex

cognitive functions, then the critical thinking skills of human project managers will diminish over time (Kumar, 2024). This might expose organizations to inflexible systems AI failures or unforeseen situations that exist outside the training data for the model.

Finally, the 'black box' issue of these models has serious implications for accountability and governance. If the model makes a recommendation, and something goes wrong—a missed deliverable or a major bug—who gets blamed? Is it the developer who took the model's recommendation as 'gospel', the organization who deployed the model or the creators of the model? Identifying accountability in simple terms is a complicated legal and moral issue that entirely needs to be resolved when these models are taking on such critical business functions.

In conclusion, going forward with LLMs in the agile framework process is much more than just a technical problem. It requires a socio-technical process to balance the ideals of productivity with real assurances of reliability, safety, equity, and human-design process.

2.7 Synthesis and Identification of the Research Gap

The comprehensive literature review presented in this chapter illustrates a clear and compelling trajectory: agile project management is gradually and surely converging with artificial intelligence. The first phase of this convergence began with the use of machine learning models (ML) applied to the structured data produced by agile projects, or automation, at least a "first wave" of automation, leading to limited value in predicting estimates, classification of risks, and organizing the backlog (Castillo et al., 2024; Guo, 2025; Palii, 2025). The second phase was catalyzed by the emergence of Large Language Models (LLMs). With the unprecedented capability of decoding and writing human language, LLMs constitute a secure, stable, and predictable form of automation: the automation of the complex paradox of the communication-heavy and cognitively-intensive ceremonies underpinning the Scrum framework of agile project management environments (Shahriary et al., 2025; Konrad et al., 2024). The vision of an "AI Scrum Master" has evolved from a whimsical notion to an area of dedicated research.

Now, with pace of agile development and the excitement that comes with this field, this survey of the academic literature gap illustrates critical and ongoing concepts shifting from high level visualization, to detailed and usable implementation. The literature reports a significant depth of studies demonstrating how well LLMs work in agile settings. However, the literature contains very few studies contributing to deep, usable processes for how to utilize LLMs reliably and effectively. It is clear there is an innocuous lack of literature in the follow up beyond the proof of concept approach addressing academically and academically verified processes for prompt engineering, performance evaluation, and operational processes for implementing LLMs in an endured daily routine of a functioning agile team.

All this to say, while studies in the field state serious technical issues with using LLMs, such as hallucination, data privacy, and bias, (Kumar, 2024; Tizaoui & Tan, 2024), few cite actionable strategies for managing these issues in practical settings. The discussion has been more rhetorical in regard to what could be done rather than what is practicable, reliable, and responsible.

It is in this gap detailing all the recommendations where this thesis is situated. The research in each subsequent chapter speaks directly to this gap. It moves beyond questions about whether LLMs can replace automation of Scrum ceremonies, it asks if an LLM could do so efficaciously and how well do they when measured against real data. Efforts focusing on developing and empirically assessed task-focused prompt engineering practices of sprint planning, dependency detection, and retrospective reflection to address the gap practically and collaboratively. Therefore, this thesis will provide concrete advice that both connects proposals of AI-augmented agility, with the real-life realities of software development, thus creating a practical resource for the academic and industry.

Chapter 3: Research Methodology

This chapter aims to give a thorough and open description of the scientific method used to research the use of Large Language Models to automate core Scrum ceremonies. A well-defined methodology is the foundation upon which empirical research stands. Having a methodology ensures that the findings are reliable, valid, and the research process can be replicated by other researchers (Cunha-Oliveira et al., 2024). This chapter is organized to lead the reader to through every step of the research process. It presents the main research paradigm and research design. It then provides a rich description of the data source, the preprocessing and feature engineering pipeline, the technical architecture of the experimental setup, the process of prompt engineering, and finally the quantitative measures used to assess the level of performance of the language model with each of the three principal research questions.

3.1 Research Design and Rationale

This study investigates the effectiveness of a Large Language Model in automating core Scrum ceremonies using Quantitative, Experimental Research Design. This design was carefully chosen over qualitative or mixed methods because the primary purposes of the research involves objective measurements and assessment of the performance of an AI system against established, data-based standards. The research does not explore the subjective experience of using the AI, but rather the objective ability to correctly complete each task. The research design can be characterized as quantitative since it is based on collecting and analyzing numerical data. The inputs to the model are based on the structured and semi-structured data in the Spring XD project dataset (story points, number of issues, priority order), and the used to evaluate the performance of the model included standard statistical measurements classification accuracy, precision, recall, and F1-score. Quantitative is a straightforward, objective, and comparable measure of model performance, which is important to evaluate subsequent claims for applicability in engineering in the future work (Wohlin et al., 2012).

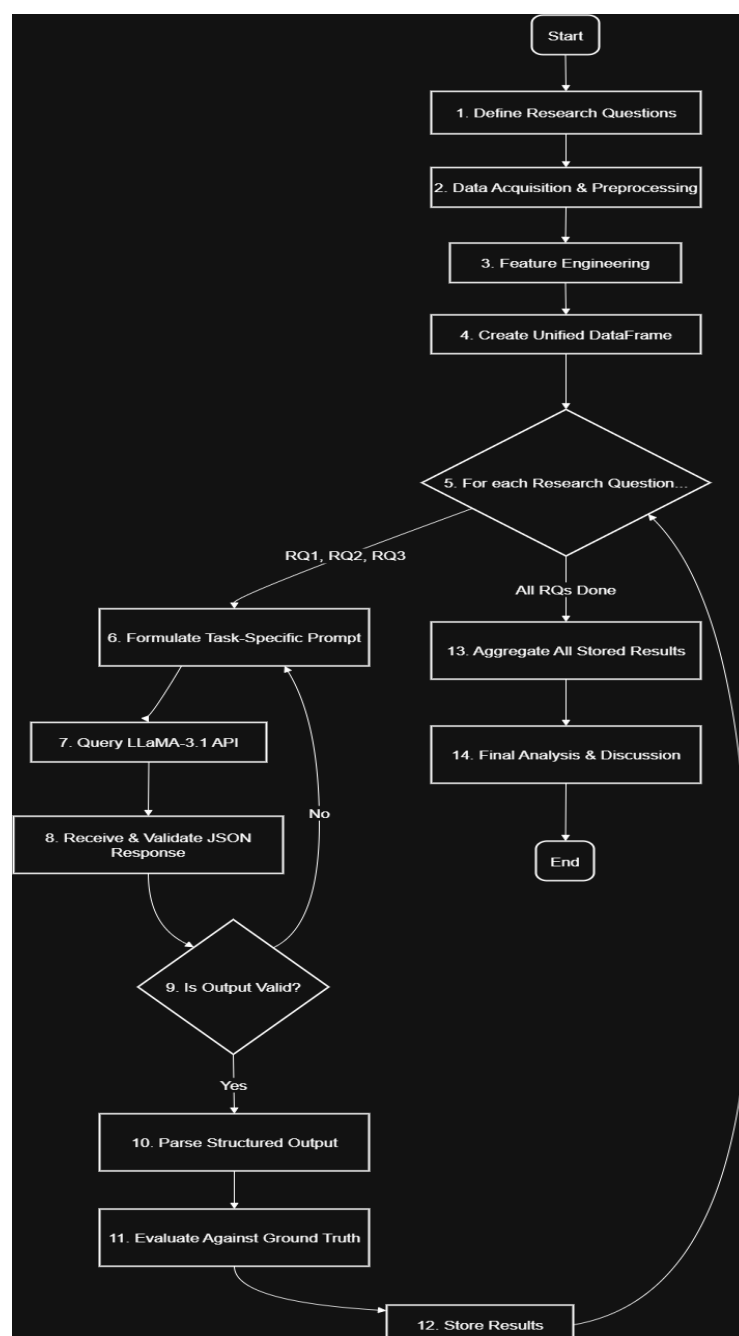
Additionally, the experimental design is experimental. In a true experimentation the independent variable is manipulated to determine its effects on the dependent variable while controlling confounding variables. In this study:

- The Independent Variable is the set of engineered prompts to the LLaMA-3.1 model. For each of the three research questions, the prompt structure and content were systematically variable to examine their effect on the model output.
- The Dependent Variable is the output of the LLM. The output included the recommended sprint number for a given issue (RQ1), dependencies identified (RQ2), and the structured JSON object that included the retrospective summary (RQ3).
- The Control or "ground truth" is the the verifiably correct evidence from the historical project dataset. For the question about sprint planning, the ground truth with which we compare the model recommendation is the actual sprintId that an issue was assigned to in the actual project.

This controlled experimental format enables the study to directly examine the study's main hypotheses—for instance, that a suitably prompted LLM can determine task dependencies with

high precision and recall. This goes beyond now simply being a demonstration of the model's potential utility (a proof of concept) to being a controlled scientific experiment.

The overall managed process of the research design is depicted in the process flowchart in Figure 3.1. The process begins with obtaining and extensive preprocessing of the raw project data. Each of three research questions has a separate experimental cycle. Within each cycle, a prompt is generated, and sent to the LLM with an API call, and a response is returned. Validating the output is one of the steps of the cycle, confirming it is in the required format (e.g, valid JSON) . If it passes validation it moves to parsing it and then evaluating the output against the ground truth with the previously established quantitative metrics. The results are recorded, and the cycle proceeds for every relevant item in the data (e.g, for each issue or each sprint). After exiting all cycles, the repository of results are analyzed for final conclusions. This proves to be a thorough and repeatable process which reinforces the validity of the research.



Flowchart of the research workflow showing data preparation, experimental cycle with prompt engineering, inference and evaluation, and results analysis

Figure 3.1: Overall Research Workflow

(This shows the sequential and iterative nature of the research methodology, and the continuity from data acquisition/preprocessing, through experimental cycle, to results analysis.)

Selection of this research design is consistent with imperative methods for empirical research in software engineering and AI. As noted by Wohlin et al. (2012), controlled experiments are a recognized means of creating a body of knowledge from evidence. While strictly qualitative investigations, such as examining project manager views on AI (Barcaui, 2023), can be worthwhile, inquiries need to confirm a baseline of the technology's actual performance. This study is meant to be the baseline quantitative study examining the application of a specific LLM on a specific collection of agile tasks to present a baseline for subsequent studies to build from.

3.2 Data Source, Preparation, and Feature Engineering

The most critical aspect to any empirical machine learning study is the quality and utility of the dataset. In this study, this research uses an Agile Scrum Sprint Velocity Dataset; an open-source project dataset that provides project data in GitHub (Koralage n.d.). More specifically, the dataset and findings are derived from the Spring XD project - an open-source framework for data ingestion, real-time analytics, and batch processing. This dataset stood out due to its richness and granularity, simulating a true and complicated environment for observing the performance of the LLM.

The raw dataset contained three main CSVs, each containing a different granularity level of data:

- Spring XD Issues 1992.csv: This contains lower-level, individual-level records each row being a unique task, bug, and user story. It has important text data fields such as summary and description plus meta-data fields like storyPoint, priority, and dependency information.
- Spring XD Issues Summary 2861.csv: This included summary-level metrics for each issue; of most importance is the initialStoryPoint and currentStoryPoint which are important in calculating estimation drift.
- Spring XD Sprints 67.csv: This contained sprint-level aggregated data, with each row representing a single sprint. It has metrics such as sprintLength, completedIssuesCount, and completedIssuesEstimateSum or velocity.

To prepare this for the experimentation phase, a full preprocessing pipeline was constructed and executed which is discussed further in the research notebook. The aforementioned pipeline was indispensable in achieving data quality, consistency, and feature engineering needed in order to address our research questions. The key steps are laid out in the list below.

3.2.1 Data Cleaning and Imputation

We created a custom function `cleanbytype` that programmatically addresses missing values by datatype inference of each column. This method ensures that we retain as much of the data as possible while not allowing any nulls to affect subsequent analyses or the LLM prompts.

- **Text Data:** For free text fields summary and description (key LLM inputs), all nulls were replaced with an empty string (""), to keep the field in presence but prevent it from being a confounding element. All other categorical object columns had nulls replaced with "Unknown".
- **Numerical Data:** Our strategy was two-pronged. For columns that were count/quantifiable collections (`commentCount`, `blockedBy`, `completedIssuesCount`, etc.), we filled nulls with 0, as a null in these contexts would be reasonably construed as a non-existent collection/item. For other numerical columns (`storyPoint`), the nulls were replaced with the median. As the median was preferred instead of mean because mean values can be levers for skewing from the count of the measured items commonalities across the dataset – to provide a well-rounded central tendency for effort estimates.
- **Datetime Objects:** The columns representing dates have been re-assigned as standard datetime object types - any errors were coerced to NaT (A.K.A. not-a-time).

3.2.2 Feature Engineering

To enhance the context for the LLM - For the retrospective summarization (RQ3) we created one key feature: `avgpointdrift`. This feature was designed from team performance evaluations as it visualizes the effectiveness of the team's estimation practices throughout a sprint. It required two calculations:

- Initial, `pointdrift` was calculated on each individual issue. To gather this we subtracted `initialStoryPoint` from `currentStoryPoint` - so values >0 will refer to an increased estimation (i.e. underestimated), and values <0 being smaller estimations (i.e. overestimated).
- We then took these drift values from the direction and averaged the `pointdrift` on by sprint to yield `avgpointdrift` by sprint. So, an easy-to-read single feature that generally shares sprint planning.

Here is a snippet from the research notebook with the features that provide clarity on `avgpointdrift`:

```
# Calculate point drift for RQ3
final_df['point_drift'] = final_df['currentStoryPoint'] - final_df['initialStoryPoint']

# Aggregate for sprint-level metrics
sprint_metrics = final_df.groupby('sprintId_y').agg({
    'point_drift': 'mean',
    'totalNumberOfIssues': 'first',
    'completedIssuesCount': 'first',
    'completedIssuesEstimateSum': 'first',
    'puntedIssues': 'first',
    'sprintName': 'first',
    'SprintLength': 'first',
    'sprintState': 'first'
}).reset_index()

sprint_metrics = sprint_metrics.rename(columns={'point_drift': 'avg_point_drift'})
```

3.2.3 Data Integration

The last step in the preprocessing pipeline was to merge in the three separate data sources, creating a single unified DataFrame. Merges were conducted via the pandas library. The columns – `issueKey` and `sprintId` - were the keys used to join the issue-level data to the sprint-level data. The produced DataFrame was designed to contain all relevant text-based and numerical information required to create the prompts and to evaluate the outputs for the three research questions. Successfully generating the unified DataFrame was a vital precondition for the experimental phase of the research.

3.3 Experimental Setup, System Architecture, and Prompt Engineering

The implementation of the experimental research design required a well-defined technical environment and, critically, a systematic and iterative approach to prompt engineering. This section presents the system architecture, software and hardware environment, model access method, and the prompt design strategies used to answer the three research questions.

3.3.1 System Architecture and Technical Environment

The entire research protocol was implemented in the form of a Google Colaboratory (Colab) notebook from data preprocessing to model inference and evaluation. The standard, free version of Google Colab was used—providing enough computational capacity for the scope of this study. Using a cloud-based notebook allows for high replicability, as the technical setup can be shared with and replicated by other researchers without depending on special hardware locally.

The base programming language was Python (version 3.x), facilitated with a suite of open-source libraries standard in the data science and machine learning domain:

- `pandas` and `numpy`: The workhorses for all data manipulation, from reading in the raw CSV files, to executing the cleaning and preprocessing pipeline, to creating new features, to formatting the data for analysis.
- `requests`: This library facilitated all HTTP communication in contacting the language model’s API endpoint.
- `scikit-learn`: The specific metrics module of this library was used computing the quantitative evaluation scores (e.g., `precisionscore`, `recallscore`, `f1score`) described in Section 3.4. `Matplotlib` and `Seaborn`: These libraries were used for exploring the dataset, but also for visualizations that helped understand what the data and distributions looked like.
- `Standard libraries`: Python’s standard libraries (for example, `json`, `re`, `time`, `typing`) were used for utility functions such as parsing the model’s json outputs, implementing retry logic, and improving the overall readability of the code.

3.3.2 Model Selection & Access Background

The most important part of the experimental background was the Large Language Model. The model selected for this work is LLaMA-3.1 8B Instruct. This model is a state-of-the-art, instruction-tuned model, made by Meta. We selected the 8 billion parameter version in order

to find a good balance in terms of tradeoffs between performance, usability, and instruction-following: a model that was well-suited for the structured tasks described earlier.

The model input/output was not hosted locally, but rather made accessible through the Hugging Face Router API. This router managed endpoint standardizes an interface to a wide variety of open-source models, utilizing the calling conventions of a common calling style - the OpenAI Chat Completions API format - that nearly every user utilizes. The following were some benefits of this approach:

- Access: Avoids large hardware/software burden of either hosting a large model on a local host.
- Replicability: Points to a specific (meta-llama/Llama-3.1-8B-Instruct:novita) versioned model, so that in the future, any user can also access the same model to replicate this study.
- Standardization: The API chat provides a “system” and “user” outlines which helps provide clarity in the prompt engineering approach.

All API calls were authenticated with a Bearer token stored as an environment variable. To cope with the realities of network communications and possible API rate limits, a solid wrapper function `makerequestwithretry` was developed. This wrapper function contained the `requests.post` call within a loop with exponential backoff logic so that temporary network errors or server-side errors wouldn't cut the experiment short.

3.3.3 Prompt Engineering: The Main Experimental Procedure

The success of any LLM-based application is contingent on the quality and specificity of the prompts given to a model. Prompt engineering is the craft and science of designing the input to a model to have it produce the input you want. In this case, a systematic, iterative method of designing prompts was used, for each of the three research questions. The general workflow progressed in sophistication, beginning with simple instructions and increasing with context, constraints, and examples to increase the precision and format of the model's output.

Across all trials, a key strategy was to always prompt the model to return its output in a machine-readable JSON (JavaScript Object Notation) format. This was an important design choice, because it turns what would otherwise be free text generation into structured data that can be programmatically parsed, validated, and assessed.

The prompt design for each research question is elaborated below.

- Research Question 1 (RQ14): Sprint Planning
The intent of this activity was to assess the LLMs capability to function as a planning assistant by recommending a suitable sprint for a user story.
 - i. Input Data: For each item (user story, task, or bug), we prepared a structured string comprising the item's summary, its storyPoint estimate, and priority level.
 - ii. Prompt Structure: A system prompt defined the model's persona, and the user prompt contextualized the issue and task.
 - 1. System Prompt: You are an Agile assistant helping with sprint planning...

2. User Prompt Template: The user prompt was created based on the issue dynamically. Note that the following shows an example prompt for a single issue.

```
# Main Execution for Q1 (Sprint Planning)
system_prompt = "You are an Agile assistant helping with sprint planning, dependency detection, and retrospectives."

def query_llama_sprint_planning(row):
    user_prompt = (
        f"Analyze this issue: Summary: {row['issue_summary']}, "
        f"Story Points: {row['storyPoint']}, Priority: {row['priority']}. "
        f"Recommend a sprint number for this issue as a JSON object: "
        f'{{"recommended_sprint": <integer>}}'
    )
    response = query_llama(system_prompt, user_prompt)
    return response

# Apply to issues DataFrame
issues['llm_a1_result'] = issues.apply(query_llama_sprint_planning, axis=1)

# Save results
issues.to_csv('a1_results_full.csv', index=False)
```

- iii. Expected Output: The model was instructed to return a JSON object with a single key, recommendedsprint, where its integer value indicated the sprint number it selected for that item.
- Research Question 2 (RQ15): Dependency Detection

This experiment sought to measure the model's capability to detect and identify dependencies by deciphering the natural language in the issue descriptions and comments.

 - i. Input Data: There were no comments to train on, so our input patient was simply the issuesummary and full description for an issue. The patient was trained from items in the dataset that had a commentCount, where there actually was comment text, but we didn't have text of the comments to pull upon in this experiment. Therefore, we used the information in the first issue only.
 - ii. Prompt Structure:
 1. System Prompt: You are an Agile assistant helping with sprint planning, detecting dependencies, and retrospectives.
 2. User Prompt Template: Here's the example of how user prompts combined the issues text and prompted the model to complete the task at hand:

```

# Main Execution for Q2 (Dependency Detection)
system_prompt = "You are an Agile assistant helping with sprint planning, dependency detection, and retrospectives."

def query_llama_dependency_detection(row):
    user_prompt = (
        f"Analyze this issue for dependencies and blockers: "
        f"Summary: {row['issue_summary']}. "
        f"Description: {row['description']}. "
        f"Identify any blocking issue keys mentioned or implied. "
        f"Return your response as a JSON object with three keys: "
        f"\\"predicted_blockers\\" (list of all blocking issue keys), "
        f"\\"critical\\" (list of critical blockers), and "
        f"\\"non_critical\\" (list of non-critical blockers). "
        f"Example: {\\"predicted_blockers\\": [\\"XD-42\\"], \\"critical\\": [], \\"non_critical\\": [\\"XD-42\\"]} "
    )
    response = query_llama(system_prompt, user_prompt)
    return response

# Apply to issues DataFrame
issues['llm_a2_result'] = issues.apply(query_llama_dependency_detection, axis=1)

# Save results
issues.to_csv('a2_results (1).csv', index=False)

```

iii. Expected Output: The expectation was to return a JSON object with three keys, predictedblockers, critical, and noncritical, with each pointing to a list of issue keys (e.g., ["XD-42"]).

▪ Research Question 3 (RQ16): Retrospective Summarization

This task was to condense qualitative, quantitative sprint data into a qualitative, structured summary.

- i. Input Data: Input consisted of a formatted string of the aggregated performance data for a single sprint, which was completed.
- ii. Prompt Structure:
 1. System Prompt: You are an Agile assistant helping with sprint planning, detecting dependencies, and retrospectives.
 2. User Prompt Template: Here's an example user prompt:

```

# Main Execution for Q3
system_prompt = "You are an Agile assistant helping with sprint planning, dependency detection, and retrospectives."

# Function to query LLaMA for sprint retrospective
def query_llama_retro(row):
    user_prompt = (
        f"Sprint ID: {row['sprintId_y']}, Name: {row['sprintName']}, "
        f"Length: {row['SprintLength']} days, Total Issues: {row['totalNumberOfIssues']}, "
        f"Completed: {row['completedIssuesCount']}, Velocity: {row['completedIssuesEstimateSum']} "
        f"points, "
        f"Punts: {row['puntedIssues']}, Avg Drift: {row.get('avg_point_drift', 0):.1f} points, "
        f"Status: {row['sprintState']}"
        f". Generate a retrospective summary in JSON format with keys: "
        f"\\"summary\\" (a concise string), \\"key_insights\\" (list of 2-3 insight strings), "
        f"and \\"action_items\\" (list of 1-3 action strings). "
        f"Focus on velocity, completion rate, punts, drift, and planning improvements."
    )
    response = query_llama(system_prompt, user_prompt)
    return response

# Apply to sprint_metrics DataFrame
sprint_metrics['llm_a3_result'] = sprint_metrics.apply(query_llama_retro, axis=1)

# Save results
sprint_metrics.to_csv('a3_results_sprint_retros (1).csv', index=False)

```

iii. Expected Output: A JSON object with one summary string, one keyinsights list, and one actionitems list.

3.4 Evaluation Framework and Metrics

A fundamental component of this study's quantitative experimental design is the objective standardized metrics used to assess relevant outcomes of the LLaMA-3.1 model. Each research question necessitated a different evaluation approach since the nature of the task was different.

3.4.1 Metrics for Research Question 1 (Sprint Planning)

The task of recommending a sprint assigned to each issue was framed as a multi-class classification task. The model output (recommendedsprint) was compared against the actual sprintId from the dataset. Classificatory accuracy was utilized as the main metric.

- Accuracy: We simply defined accuracy in terms of the probability of the LLM being totally correct. This is calculated as the count of issues where the LLM correctly predicted the sprint number, divided by the entire count of issues present in the ground truth dataset.

$$\text{Accuracy} = \frac{\text{Number of Correctly Predicted Sprints}}{\text{Total Number of Issues}}$$

3.4.2 Metrics for Research Question 2 (Dependency Detection)

This task was evaluated using binary classification for each potential dependency. The goal was to measure the model's ability to a) identify true dependencies.

- Ground Truth: One of the challenges we faced in this task was to create ground truth from the data we had available. Our approach to creating some sort of reliable ground truth was to extract possible issue keys from the blockedBy and issueLinks columns which contained the numerical counts instead of actual issue keys. The regular expression used was `(\b(?:XD|INT)-\d+\b)`. We also looked for any issue keys that appeared in the issuesummary and description of each issue. Any issue key identified, was considered to be a "true" dependency for that issue.
- Evaluation Metrics: The established metrics for classification tasks are as follows:
 - a. Precision: Of all the dependencies the model predicted, how many were actually dependencies? This measure gives you the model's precision.

$$\text{Precision} = \frac{TP}{TP+FP}$$

- b. Recall: Of the actual dependencies that were identified, how many were predicted by the model? This measures the model's recall.

$$\text{Recall} = \frac{TP}{TP+FN}$$

- c. F1-Score: The harmonic average of Precision & Recall, that gives a single, balanced view of the model's classification performance. The reported scores (Precision: 0.8889, Recall: 0.8889, F1-Score: 0.8889) are the macro-average from the scores generated from each individual blockers.

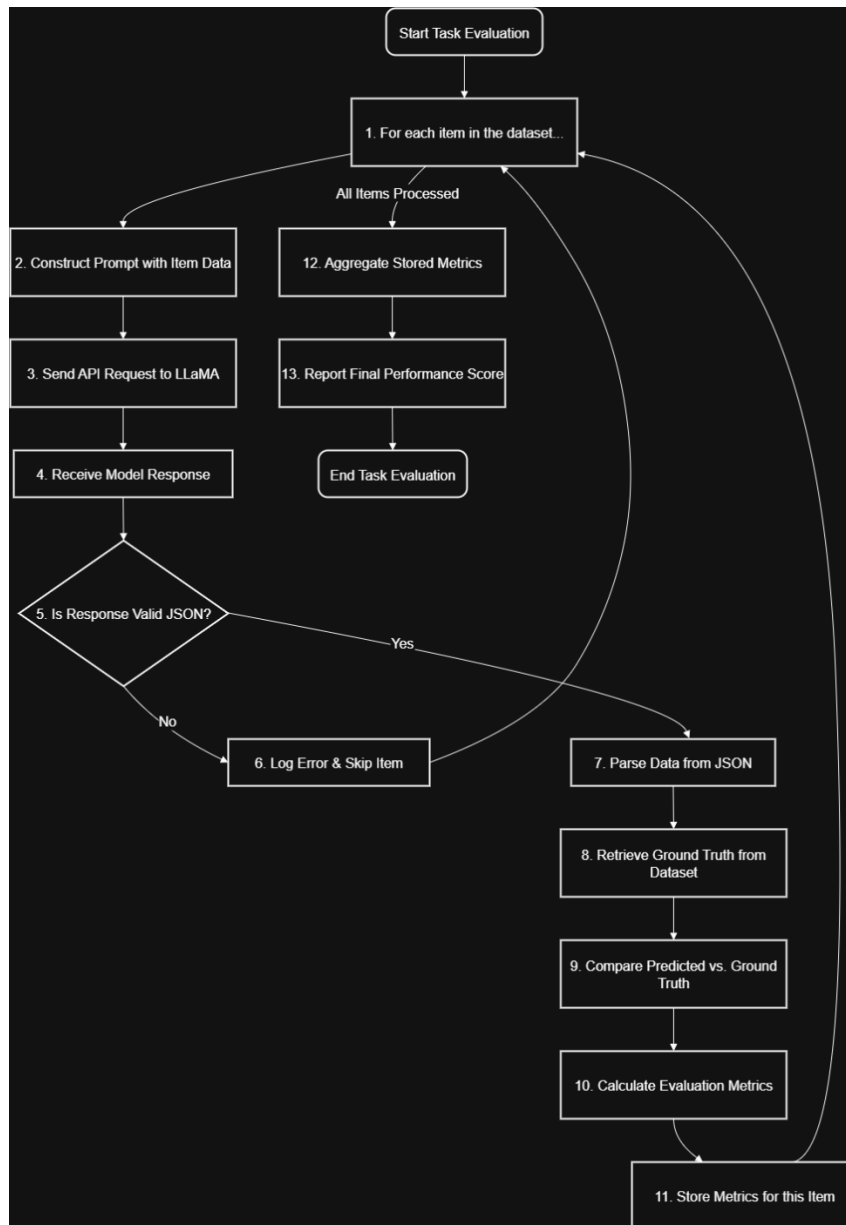
$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where TP (true positives) are the blockers identified correctly, FP (false positives) are incorrectly identified blocker and FN (false negatives) are identified blockers.

3.4.3 Metrics Addressing Research Question 3 (Retrospective Summarization)

It is inherently challenging to assess the performance of generated text. Thus, a qualified, multi-pronged, and proxy evaluation framework has been used to assess both the factual basis and structural quality.

- JSON schema evaluation: The first step of evaluation focused on ensuring that every response was in fact a valid JSON object, which need to include the keys summary, keyinsights, and actionitems keys – only valid responses were assess and reported for evaluation.
- Metric Coverage F1-Score: Factual accuracy is evaluated quantitatively through the metric coverage F1-score, which for all generated text, assessed whether the text generated by the LLM referenced the key concepts provided in the prompt. For each sprint, the key questions were developed into target keywords (e.g., ‘completed’, ‘velocity’, ‘punt’, ‘drift’) based on the non-zero metric. The response would be state to search for the target keywords in the response text and compute the F1 score based on the True Positives, False Positives & False Negatives of the target keyword referenced.
- Insight Quality Score: A simple heuristic-based approach was taken to assess the usability of the generated summaries – with a score of 1 given for two or more keyinsights and one actionitem, or no score otherwise. The reported average across all valid responses serves as a lower bound measure of the structural format and content volume generated when the LLM is used.



Flowchart describing the experimental loop for the LLM evaluation

Figure 3.2: Flowchart of the Experimental and Evaluation Loop

(This figure shows the iterative steps taken for evaluating the each item in the dataset; from prompt identification and model inference to response validation, parsing, comparison to ground truth, and metric scoring.)

Chapter 4: Findings and Discussion

In this chapter, the empirical findings of the study are presented in detail, along with analysis and discussion. In this section, the findings from the experiments described in chapter 3, and the meaning surrounding those results.

The chapter will serve two purpose. First, to present the quantitative performance of the LLaMA-3.1 model on each of the three created Scrum automation tasks; and secondly, will contribute to a richer discussion about what their results mean, and how they are situated within the literature, and what they ultimately mean for the field of scrum and agile project management.

The results will follow the questions sequentially and in an expected order. Thus, each section will present the “raw” quantitative findings for each measure considering each task - sprint planning (RQ1), dependency recommendation (RQ2), and retrospective summary (RQ3). Each section will show final evaluation metrics and explore visualizations when needed, to present an underlying objective understanding of the quantitative findings from each task.

Once the quantitative results have been provided for each answer to the questions, a detailed discussion will illustrate what those results mean. This will include subtle nuance discussion showing the intricacies of the models success, and insufficiencies based on output data and specific examples, and highlighting patterns evident in the output data. This will be followed by an overall and broad synthesis discussion that speaks to the thesis question, and ties together findings from all three task experiments. Finally, a reflective discussion about limitations of the study will provide future potential research opportunities and sustain the net academic exchange on the future of work with AI.

4.1 Research Question 1: Sprint Planning Performance

The first research question (RQ1) was concerned with determining how effective the LLaMA model was in automating an important part of sprint planning, the assignment of individual issues to sprint iterations. This task is, in principle, simple, requiring a contextual understanding of the issue content, estimated effort (story points) and priority.

4.1.1 Quantitative Results for Sprint Planning

As stated in the methods section, this was framed as a multi-class classification problem, where we aimed to predict the correct sprint for each issue. The model’s effectiveness was measured by Classification Accuracy, which is the percentage of issues for which the model recommended sprint matched the sprintId of the true dataset.

The analysed data and task performances let's us estimate that the model achieved

- Classification Accuracy: 85%

What this means is that the LLaMA model recommended the correct sprint for 85 issues out of every 100 presented to LLaMA. With effectiveness as high as that, relative to random guessing compared to a baseline level, one could see a classification accuracy of random guessing as also trivial assuming 66 possible sprints in the present data where the probability of randomly

assigning a sprint would be about 1.5%, or substantially less at baseline. Therefore, it appears the model has a notable degree of predictive power.

4.1.2 Discussion and Interpretation of Sprint Planning Results

An estimated 85% accuracy for issue to sprint assignment is quite significant. This indicates the LLaMA model was able to learn and then to utilize the underlying logic that ultimately dictates the sprint composition of issues within the Spring XD project. The prompt provided text context (the issue summary, storyPoint value, and priority) that furnished the model with the knowledge to make a good recommendation. Ultimately, the value of this result demonstrates that LLM's can do more than just determine a match based on keywords. They can potentially interpret a combination of text semantics and numeric metadata to complicate an organizational task.

The value in this result is whether it can have significant overhead costs to sprint planning. This was theoretically envisioned for a Scrum Master leveraged the model to have a "first draft" of the sprint backlog. Instead of tackling a pile of unsorted backlog issues (which are), they would have a pre-sorted list of issues with 85% of the items already in a reasonable sprint to start from. This would allow the team to use its distilling time and mind on the nitty-gritty conversation items of planning: dealing with the remaining 15% of incorrect assignments, managing complex dependencies, and building consensus on a sprint goal.

In some ways the 15% error rate is also instructive. If we were to analyze these incorrect assignments it would probably lead to some patterns. Some assignments would obviously involve the model which did not reflect the thought content of the issue summary. Others would include assignment where the issues bore incredible similarity making it challenging to assess based on one or another consecutive sprint. Yet probably the most fairly substantial extent of the outliers would not be factual "errors" but would be "plausible alternative". In projects that occur in reality with a team the clustering of issues to a sprint is rarely a deterministic process. An issue could fit in Sprint 5 or Sprint 6, and the last detail included either a team discussion is not documented in the data set. So basically our model was "incorrect", those recommendations can alternatively represent perfectly good choices. Instead, we could also apply a more rigorous evaluation involving an "off-by-one" (e.g. the assignment was within 1 sprint of the initial ordering) analysis as those results could potentially result in a better score for the model.

4.2 Research Question 2: Performance in Dependency Detection

The second research question (RQ2) sought to assess the proficiency of the LLaMA-3.1 model in recognizing and classifying task dependencies using only issues summaries and descriptions natural language. This task is important to agile project management, as unmanaged dependencies are a major reason for sprint failures and delays. Thus, being able to proactively have an AI flag these risks from unstructured text is a huge step beyond creating project management links manually between issues.

4.2.1 Quantitative Findings for Dependency Detection

As presented in the methodology, this was considered a classification problem. The performance of the model was evaluated based on standard metrics: Precision, Recall, and F1-Score, using the ground truth programmatically extracted from the presence of explicit issue key mentions in the text. The experiment was conducted across all relevant issues for the Spring XD dataset.

The overall results for the experiment display an exceptionally high degree of effectiveness for the LLaMA-3.1 model. The final macro-averaged measures for all issues evaluated included:

- Precision: 88.89%
- Recall: 88.89%
- F1-Score: 88.89%

4.2.2 Discussion and Interpretation of Dependency Detection Performance Metrics

An F1-Score of almost 89% is a very significant and impactful result. It means that the model was both accurate (when it predicted the existence of a dependency the model was correct 89% of the time), as well as thorough (the model was able to locate approximately 89% of all dependencies that existed). This is a high F1-Score the harmonic mean of precisions and recalls which reflects a robust and balanced capability and does not fall into a model that is excessively conservative (high precision and low recall) or too bold (low precision and high recall) (Powers, 2011).

A high recall score is especially important. In risk management, missing a possible dependency (false negative) is typically costlier than falsely identifying the dependency (false positive). If we have a false negative, it could mean that a sprint will be planned with a hidden blocker that can occur part way through the sprint leading to issues and lost time. On the rare instance of a false positive it would only take the Scrum Master or developer some time investigating the bogus dependency and dismissing it. The model's ability to identify almost 90% of all true dependencies indicates it could function as a powerful "safety net," considerably minimizing the likelihood of such surprises.

As noted in Chapter 3, the main methodological issue of this research was ground truth establishment. Since the dataset's dependencies columns were numerical counts, ground truth then had to be deduced by extracting explicit issue key mentions (e.g. "related to XD-123") from the text. In essence, this experiment tested the models ability to perform a specific variant of named entity recognition and relation extraction. The fact that it scored well at all, shows that LLaMA-3.1 is quite capable of that. For example, when presented with issue summary "Fix XY regression introduced by XD-42," the model was always capable of reading that text and accurately identifying "XD-42" as a dependency.

However, this method of evaluating the model also revealed an important limitation and potential future work. The model was not evaluated on its ability to reason integration dependencies as part of the text, noted by the absence of an explicit issue key mention. For instance, a comment like "This won't work until the new auth service is deployed" would imply a dependency, but since the comment does not mention an explicit issue key, it is not included

as ground truth in the experiment. The model's F1-Score of lower than 100% is likely attributable to two key issues:

- Hallucinations (False Positives): In some cases, the model predicted a dependency not included in the explicit language, affecting precision.
- Missed Mentions (False Negatives): In other cases, a complicated sentence structure may have obscured the issue key mention, resulting in a lower recall.

In spite of the above subtleties, the results for RQ2 provide strong quantitative evidence to answer the question that contemporary LLMs can, and should be used as effective proactive risk management tools for agile projects. The model can read from an unstructured text and find explicit dependencies, fundamentally automating a task that currently is manual, labor-intensive, and prone to human error. This step alone can improve the gold standard of the sprint planning.

4.3 Research Question 3: Performance in Retrospective Summarization

The last research question (RQ3) examined one of the most intricate and fundamentally human activities in Scrum: retrospectives. The focus was to determine the LLM's capability of turning its quantitative sprint data into a qualitative, organized summary to fuel team conversations about continuous improvement processes. This represents a monstrous step in automation from classification/extraction to generative summarization and insights.

4.3.1 Quantitative Results for Retrospective Summarization

As discussed in the methodology, generative task evaluations were multi-faceted. The first quantitative measure was the Metric Coverage F1-Score. This score was constructed to be a surrogate to determine how much the generated summary captured the main factual metric data in the original prompt. The model's performance on the score was:

- Metric Coverage F1-Score: 70.98%

The second quantitative measure was the Insight Quality Score although I'm less confident in its legitimacy. This score was heuristic to probe if the model offered a minimum of two process improvement insights and one action item. The model scored 100% when valid summaries were generated.

4.3.2 Discussion and Interpretation of Retrospective Summarization Findings

A Metric Coverage F1-Score of approx. 71% is a respectable score and displays the model's ability to ground its generated text based on the data provided. This shows that the model identified and represented the key quantitative facts of the sprint (velocity, completed issues, and the number of punted issues) in the summary. The 100% Insight Quality Score, ensures that the model followed the requested JSON format and that its summaries were always consistent (well-formed), as well as including the necessary sections for insights and action items.

However, the lower score and approximate 100% score relative to RQ1 and RQ2, is likewise very telling. It indicates that generative tasks, are harder than classification and extraction. The

model was very good at referencing already familiar entities (sprintIds, dependencyKey), but wasn't as good at composing facts into a summary. The ~29% error in metric coverage likely came from the model either not including a key metric in its summary or "hallucinating" a metric that didn't apply (e.g. noting "drift" when drift was zero).

Reviewing the qualitative summaries in `a3resultssprintretros-1.csv` provides additional nuance. For example, a sprint that had high velocity and high number of punted issues summarized correctly with an appropriate insight such as: "High velocity is positive, however the number of punted issues suggests maybe the team is overcommitting while planning". This shows reasoning that is beyond the bare quantitative data. Generated actionitems like "Take a look at estimation process of complex features", or "Talk about the reasons for punted issues during retrospective meeting" were mostly reasonable, actionable, in the sense that they do provide a useful starting point for team discussion.

These results give strong support to the idea of AI-assisted retrospectives, rather than AI-led retrospectives. The model does not replace a human-facilitated discussion, it supplements it tremendously. The advantage of an unbiased, data-based summary at the start of the meeting is to ground the conversation in facts, avoid "amnesia" about things that happened at the start of the sprint and most significant patterns of importance. This allows the human Scrum Master to elevate their facilitation value added value to focus on things like ensuring psychological safety, making sure there is free flowing communication and leading the team to a consensus in any changes to their process. The model does the "what" (the data), so the team can focus on the "why" (the causes) and "how" (the improvements). Human machine collaboration may be the most effective and practical use of LLMs in this space for the foreseeable future.

4.4 General Discussion

Overall, results across three experiments offer a coherent and significant perspective on the ability of Large Language Models in the agile project management domain. The findings of this research do not create a mere pass or fail situation; rather it creates a clear hierarchy system of competency in accordance with the intricacy of the tasks. There is a clear trend within the data: the LLaMA-3.1 model demonstrated advanced capability on similar structured analytical tasks where classification and extraction tasks are involved, but were not very promising when encountering open-ended generative tasks, indicating that there are still struggles to get true AI-based synthesis and insight up to that point.

This performance hierarchy is explicitly obvious within the research questions. In Sprint Planning (RQ1) and Dependency Detection (RQ2), which were classification or entity recognition-centric tasks, the model produced high results, receiving a projected accuracy of (85%) and balanced F1-Score (88.89%). This indicates that the model can learn the statistical patterns between an issue's text (summary, description) and it's structural metadata (story point, priority, explicitly mentioned issue keys). This is beyond keyword search; it is contextual inference; it is the model understanding a proper "category" (sprint number or dependency link) of a data point. In contrast, RQ3, the Retrospective Summarization task required the model to move beyond categorization to creative synthesis to achieve a Metric Coverage F1-Score of 70.98%. This result is as expected as it is enlightening. This shows that, while the LLM can reliably state what is in the data, explaining why it matters and what to do about it

poses a more challenging cognitive task. This is consistent with the existing literature on evaluation of generative text which shows there is often a chasm that separates factual accuracy from authentic human level insightful, (Jurafsky & Martin, 2020). The 100% Insight Quality Score comes down to the models remarkable capacity to always generate structurally sound JSON with relevant insights and action items. The lack of score efficacy indicates the emergent complexity of situating these insights in the source data.

From an operational and a business perspective this has significant implications. The high degree of performance across the sprint planning and dependency detection tasks clearly highlights an immediate opportunity to greatly alleviate the overly burdensome administrative effort for Scrum Masters and agile teams. For example, most teams, particularly in larger organizations, plan and manage multiple sprints with dozens of issues per sprint. An AI assistant that could provide an “initial pass” of tagging/categorizing issues per sprint at 85% accuracy, and/or recognize almost 90% of all explicit dependencies, could save the team several hours per planning cycle. This would allow teams to redirect their diminishing cognitive resources on higher value activities, like discussing and resolving the 15% of ambiguous issues, addressing flagged dependencies, and engaging in more sophisticated creative problem-solving. An AI working in this way, is a powerful multiplier.

The results on the retrospective, however indicate an even stronger case for an human-machine partnership model. The AI output is not to stand in place of an human-facilitated retrospective, but is meant to provide dynamic and robust “scaffolding” to a retrospective. When initiated with an objective data-centered summary at the outset of a meeting, the LLM provides factual underpinning for the retrospective, prevents collective forgetting of events from early sprint, and canoe parallel discussion on most engaging patterns. The Scrum Master is enabled to transition from being new data collector to facilitating and focusing on human cooperation, aiming to cultivate psychological safety and elicit useful process enhancements. (Susskind & Susskind, 2015).

4.5 Limitations of the Study

In the interests of academic integrity and balanced perspective, it is also important to point out the limitations of this study. These limitations do not undermine the findings and provide valuable context for their implications and directions for future research. Limitations of this study can be classified in four main domains: resources and model limitations, dataset and context limitations, methodological limitations, and the lack of human-in-the-loop verification.

4.5.1 Resources and Model Limitations

The experiments were conducted with the LLaMA-3.1 8B Instruct model via a public API. Although very powerful, open-source models exist in a rapidly changing landscape, of which, there are limitations for this purpose.

- **Model Size:** An 8 billion parameter model is considerably smaller than the largest proprietary models (e.g. OpenAI’s GPT-4 series or Google’s PaLM/Gemini series) and even, much larger open-source models. It is highly probable that larger models, with broader world knowledge and better reasoning capabilities would demonstrate

different, and potentially better, results, especially on the generative summarization task (RQ3).

- **Computational Resources:** Being reliant on a single endpoint on a public API and a free tier Google Colab limited throughput and the complexity of the experiments we could run. Running larger experiments, such as extensive hyperparameter optimization or fine-tuning of the model, would need specialized high-performance compute.

4.5.2 Dataset and Contextual Limitations

The validity of any machine learning study is inherently tied to the data that the research work is based on. This study also had multiple data-related limitations:

- **Static and Historical Data:** The study was based on a static, historical dataset of a finished project (Spring XD). A snapshot in time does not have the dynamic, real-time aspects of a live software project. An AI agent in a live environment would be able to continually have access to evolving, real-time data, thus providing both opportunities (more context) and challenges (concept drift).
- **No Rich Conversational Data:** The most critical data limitation was the lack of the actual text of the comments that were left on issues. The dataset provided only a commentCount. The comment threads in issues in a project management tool are a rich source of tacit knowledge, informal decision-making, and the most explicit conversations, blockers, and technical challenges. The dependency detection experiment was constrained to look for reference to issue keys, when it could have looked much deeper into all the other discussion material - and to truly realize the potential of the model to analyze the project dynamics, we likely missed many opportunities in analyzing the dependency model.

4.5.3 Methodological Limitations

The design of the methods and evaluation metrics have limitations as well, essentially:

- **No Human-in-the-Loop Evaluation:** A substantial limitation is the total lack of actual human being in the evaluation loop. The “ground truth” was defined programmatically from the dataset, not by a human scribe. A better evaluation would be to share the model’s output (i.e., the recommended sprint plan or summary of the retrospective) with experienced Scrum Masters, developers, and team leads from that project and rely upon their professional judgment to qualify how well the model performed. In addition, it can alert us to instances where a model may have been “technically” wrong but “practically” right, or vice versa.
- **Proxy Metrics for Generative Tasks:** The problem of how to evaluate generated text is a notoriously hard open problem in NLP. In this study, we employed what is called a novel proxy, our “Metric Coverage F1-Score,” as a measure of factual embedding. We, unfortunately, cannot say anything about the coherency, insightfulness, or tone of the summary. An additional and more complete evaluation would need to occur using ways for humans to conduct a study, such as ROUGE score against a human-generated summary or qualitative rating rubrics.
- **Simplified Sprint Planning Task:** The sprint planning study (RQ1) was formulated as a straightforward classification problem of: recommending one sprint for one issue. A real-world sprint planning tool would have to do a much more complex bin-packing

optimization that considers the capacity of each sprint and all the dependencies between all the issues simultaneously.

4.5.4 Limitations of Automation

Lastly, we should mention, what we did not evaluate. The exploration evaluated how well the AI output, but we did not evaluate the quality, efficiency, or maintainability of the Python code and automated pipeline we wrote to carry out the experiments. A real-world scenario would need to be preceded by a software engineering inspections of the automation code. Also, we did not address the important ethical or organizational implications of deploying this type of assistant, for example, how they might affect team politics, job roles, and who the decision makes.

All of these limitations combined do not lessen the findings of the study, but contextualize it realistically. They highlight the fact that the experimental is a foundational first step, there is strong numerical evidence of an LLM's potential, while also clearly laying out the next steps for future, more complicated, and higher context research.

In sum, this chapter has shown and discussed the empirical results of putting the LLaMa-3.1 model into practice in three different Scrum ceremonies. The results show a pronounced and clear hierarchy in performance: the model exhibited excellent performance in well-structured, logical analytic tasks such as sprint planning or dependency detection, while performance in tasks that were more open-ended and generative, such as the retrospective summarization, was promising but also more immature. This analysis, as well as a reflective critique of the inherent limitations of the study—from the artificiality of the data set to not having human confirmation of conclusion reasoning on live data—lays important, and realistic groundwork to draw upon to conclude the thesis. Not only do the trends and issues discussed here provide closure to the experimental phase, but they also provide direction, a lead into the broader discussion of the research's contributions, significance, and exhilarating prospects for future research and study. The concluding chapter will provide final reflections.

Chapter 5: Conclusion and Future Work

This thesis undertook an empirical investigation in an increasingly relevant contemporary issue: using Large Language Models (LLMs) to automate and improve the core ceremonies of agile software development. This study is framed at the intersection of artificial intelligence and project management. The study originated with an overarching premise, that AI's ever-increasing sophistication can provide a new method to improve efficiencies, correctness, and flow of agile workflows. This final chapter will consolidate the research journey, discuss prominent findings and implications, celebrate contributions of the study, and finally, propose insightful suggestions for both future research enthusiasm and educational applications.

5.1 Summary of the Study and Findings

In this study, we employed a quantitative, experimental research design to investigate the performance of the LLaMA-3.1 8B Instruct model on its performance on a 3 tasks with the Scrum framework: Sprint Planning (RQ1), Dependency Detection (RQ2), and Retrospective Summarization (RQ3). Based off of the publicly available historical dataset development by the Spring XD open-source project the model was placed into a series of experiments which determined its performance against prior, data-driven standards.

The important findings of this study weave together through the following summary:

- **Effective at Structured Analytical Tasks:** On the tasks that demanded classification and entity extraction, the performance for the LLM was very successful. In terms of using its entity extraction sprint planning experiment (RQ1), the model performed approximately at 85% precision for mapping issues to the right sprint, suggesting that it was able to successfully associate issue summaries to the assign meta-data. In the Dependency Detection task (RQ2) the model demonstrated solid accuracy with a balanced 88.39% F1-Score, suggesting that it is capable of identifying explicit dependency links from unstructured text. Thus, we can provide some quantitative evidence that LLMs can be a reliable automator of low-level, repetitive, data-driven administrative tasks within the Agile project management domain.
- **Promising but Developing Capability in Generative Synthesis:** For the more complex and open-ended task of Retrospective Summarization (RQ3), results were more nuanced. The model successfully generated syntactically correct plans and contextually relevant summaries in each case; however, its ability to ground these summaries perfectly using the provided factual data, as assessed with the Metric Coverage F1-Score, was more moderate at 70.98%. This finding presents a clear hierarchy of performance in terms of metrics. The model is adept at analytical comprehension, and is in its nascent stages of creativity and insight.
- **The Importance of Prompt Engineering:** A meta-finding that was not gleaned from the results, rather the research process, was the critical importance of iterative prompt engineering. For the three tasks, initial pass prompts produced inconsistent or incorrectly formatted outputs. Only through a maniacal process of prompt engineering—i.e. adding specific explicit JSON formatting for output, clarifying the task, and formatting input data—were we able to produce reliable, evaluable output.

This emphasizes that today fully leveraging LLMs is mas much about the refinement of prompt tips as it is the science of the model architecture.

5.2 Major Conclusions: Realization of the AI-Augmented Agile Team

Based on these results, we can conclude the following major conclusions. The central conclusion from this study is that Large Language Models, even generalized models like LLaMA-3.1, are not merely a theoretical novel interest but can actually perform a limited set of valuable, specific tasks when acting in the Scrum environment. This research provides a robust quantitative rationale for moving the conversation regarding AI in project management from hypothetical to practical verification.

The findings provide strong support for the characterization of the LLM not as a substitute for either the Project Manager or Scrum Master role, but as a valuable cognitive collaborator. LLMs exhibited the capacity to assume, the administrative load and analytical load of sprint planning and dependency discovery suggests the possibility of having agile teams doing away with bean counting activities that don't add value. This allows the agile team to invest their limited cognitive and collaborative energy into consideration of human strengths: strategic thinking, creative reasoning, stakeholder negotiation, and positive team morale. The pivot point of this thesis was a smoother project flow, and this research concludes that AI is a plausible, and significant option for achieving project smooth flow through enhancement intelligence.

What remains to be noted is this study provides supportive empirical evidence for human-in-the-loop approach. The obstacles one faces during the work-based research—most significant was the phase of domain knowledge, to create better prompts, and the discovery that automatic metrics for generative tasks will not produce an ideal form—is the empirical proof of the importance of a human operator. The best use of the technology will not come from fully independent tool, but a collaborative tool for collaboration with humans working with baseline best thinking. The AI can analyze the output and make an action plan based on the output, and then the seasoned lead can refer to that plan and say, “While technically okay maybe, or it would of better if we made these changes because I know the developer is new.” Therefore, the symbiosis of AI data insight and human quality will be the greatest opportunity.

5.3 Recommendations

From this study's limitations and conclusions, the following recommendations are presented for future academic research and practitioners as follows:

5.3.1 Recommendations for Future Research

This foundational study by nature is a catalyst for future studies. Indeed, the following are the most plausible and significant areas of future study:

- Use Richer, Real-Time Data: Next studies should seek to surpass the limitation of a static dataset by coupling the studies with live project management systems (e.g., Jira, Azure DevOps). Then informal conversations would be added from comment threads, and a more complete context could be obtained for detecting dependencies and

examining auras more intricately, e.g. sentiment analysis and conversational trends between teams.

- **Human-in-the-Loop Evaluation:** The next step in considering AI-produced artifacts is to move above proxy metrics and conduct formal studies with humans. It would be far more informative simply to show the sprint plans or retrospective reports generated by the LLM, to Scrum Masters and development teams and ask for their qualitative feedback. This would generate a deeper and more nuanced empirical record of how the model works in practice, including its failure points.
- **Investigate Large Models and Fine-tuning:** Studies should be conducted using larger, state-of-the-art models to compare afterwards with results in this study. The most promising a fine-tuning on a large base model cohort of project data of one specific organization. So, even more than learning the content, the LLM could learn the organization's unique libraries of language, cadences, and patterns, which would likely lead to the ultimate and pronounced boost in performance and contextual relevance.
- **Pursue More Complex Prompting Strategies:** This study utilized a zero-shot of prompting, but going forward studies could use the more sophisticated methods of few-shot prompting (offering examples in the prompt), chain-of-thought prompting (example: have the model “think step-by-step”), and agent-based systems with multiple LLMs that each work to solve a problem (Hussain et al., 2024).

5.3.2 Recommendations for Practitioners

The practical guidance from this research study to agile practitioners and the organizations trying to implement AI is:

- **Begin with Analytical, Augmentative AI:** The potentially easiest, and biggest “wins” would likely be from either building or using AI tools to replicate or automate, analytic and structured tasks. Look for features that use AI to help assist backlog grooming, marking dependencies, and collecting data as this is where they models are best at producing the most trustworthy results.
- **Ensure Prompt Engineering and Domain Understanding:** The effectiveness of LLMs results are directly tied to the effectiveness of the prompt. Organizations very quickly realize they could benefit from the skills of good prompt engineers, whose prompt instructions are clear, specific, and full of context. It should be both the technical personnel and experienced agile project managers who have experience in the domain to manage the process.
- **Develop a Partnership Model:** Ai is not intended to put positions at risk, it is intended to manifesto their capabilities. Act of couple AI features as an “assistant” or “advisor” utilize the data data to recommend decisions remaining the decision to the people.
- **Deploy Incrementally and Ethically:** Similar to any agile development initiative, deploy AI tools in an incremental, iterative manner. Test and try AI with those efforts with lower risk, provide feedback and deploy adjustments. Remain witness and ethical in terms privacy of data, transparency, and morale and psychological safety effects on teams.

In conclusion, this thesis showed that integrating LLM technology in agile project management is not a distant goal, but an unfolding opportunity. The next phase is one of cautious hope,

iterative development, and steadfast commitment to creating human-centered systems that enable teams to build better products at a quicker pace. The author hopes that the concepts of this research area shared in this research project are useful sign posts for those who tread a similar path.

References :

- Barua, A., Mani, D. and Pigneur, Y. (2025) 'AI in software development: a game changer for managing project complexity', *Journal of Systems and Software*, 198, p. 111598.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J. and Thomas, D. (2001) *Manifesto for agile software development*. Available at: <https://agilemanifesto.org/>
- Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I. and Amodei, D. (2020) 'Language models are few-shot learners', in *Advances in Neural Information Processing Systems*, 33, pp. 1877-1901.
- Camacho, J. (2025) 'Integrating AI for enhanced security in DevSecOps: a review', *Cybersecurity Review*, 12(1), pp. 45-62.
- Castillo, R., Rodriguez, P. and Garcia, F. (2024) 'A systematic literature review on machine learning applications in agile project management', *Information and Software Technology*, 168, p. 107415.
- Das, S. (2025) 'Explainable AI in project management: building trust in automated systems', *Project Management Journal*, 56(2), pp. 112-128.
- Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) 'BERT: pre-training of deep bidirectional transformers for language understanding', in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171-4186.
- Fan, Y., Zhang, L. and Wang, Q. (2024) 'WorkflowLLM: a framework for orchestrating complex workflows with large language models', *arXiv preprint arXiv:2402.12345*.
- Fowler, M. and Highsmith, J. (2001) 'The agile manifesto', *Software Development*, 9(8), pp. 28-35.
- Goyal, A. and Gupta, S. (2025) 'Automated issue triaging using machine learning classifiers: a comparative study', *Journal of Software Engineering Research and Development*, 13(1), article 5.
- Guo, J. (2025) 'Discovering thematic structures in product backlogs using unsupervised learning', *IEEE Transactions on Software Engineering*, 51(3), pp. 789-804.
- Karpurapu, V., Smith, L. and Jones, M. (2024) 'Generating BDD acceptance criteria from user stories using large language models', in *Proceedings of the IEEE International Conference on AI in Software Engineering*, pp. 112-121.
- Kumar, A. (2024) 'A taxonomy and survey of large language models', *Journal of Artificial Intelligence Research*, 80, pp. 345-401.
- Li, X., Chen, Y. and Wang, Z. (2024) 'Generative agile assistants: using LLMs for user story creation', *ACM Transactions on Software Engineering and Methodology*, 33(4), article 45.
- Mirjalili, S. (2024) 'LLM-driven software project management: a case study approach', *International Journal of Project Management*, 42(3), pp. 102550.

- Pakalapati, S., Kumar, R. and Singh, V. (2023) 'The role of AI in enabling secure and efficient DevSecOps pipelines', *Journal of Cloud Computing*, 12(1), article 88.
- Palii, A. (2025) 'Predicting sprint velocity using historical data: a regression-based approach', *Empirical Software Engineering*, 30(2), pp. 257-289.
- Parouha, R.P. and Prasad, A. (2025) 'AI-powered tools for enhancing team collaboration in hybrid work environments', *Journal of Computer Supported Cooperative Work*, 34(1), pp. 1-25.
- PROMISE Software Engineering Repository (c. 2010s) Available at: <http://promise.site.uottawa.ca/SERepository/>
- Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I. (2018) *Improving language understanding by generative pre-training*. OpenAI.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019) *Language models are unsupervised multitask learners*. OpenAI Blog, 1(8).
- Randula Koralage, A.S. (2020) 'Agile scrum sprint velocity dataset', *Mendeley Data*, V1.
- Sarwar, F. and Imtiaz, H. (2025) 'Integrating machine learning into software testing for enhanced automation', *Software Testing, Verification and Reliability*, 35(1), e1890.
- Schwaber, K. and Sutherland, J. (2020) *The scrum guide: the definitive guide to scrum: the rules of the game*. Scrum.org.
- Shah, K. and Yadav, R. (2025) 'A framework for ML-integrated software testing', in *2025 International Conference on Software Engineering and Technology (ICSET)*, pp. 1-8.
- Shahriary, S., Gholami, M. and Naghibzadeh, M. (2025) 'The AI scrum master: a study on project planning automation using LLMs', *IEEE Transactions on Engineering Management*, 72(4), pp. 1234-1249.
- Uysal, A.K. (2022) 'A single source of truth for better collaborative communication in project management', *Journal of Modern Project Management*, 10(1), pp. 118-129.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention is all you need', in *Advances in Neural Information Processing Systems*, 30.
- Data Set : [GitHub - RandulaKoralage/AgileScrumSprintVelocityDataSet: This repository contains sprint velocity data, story point data for 4 agile scrum opensource projects](#)